Go 1.24 | MCP Protocol | Multi-LLM

AI Infrastructure Agent

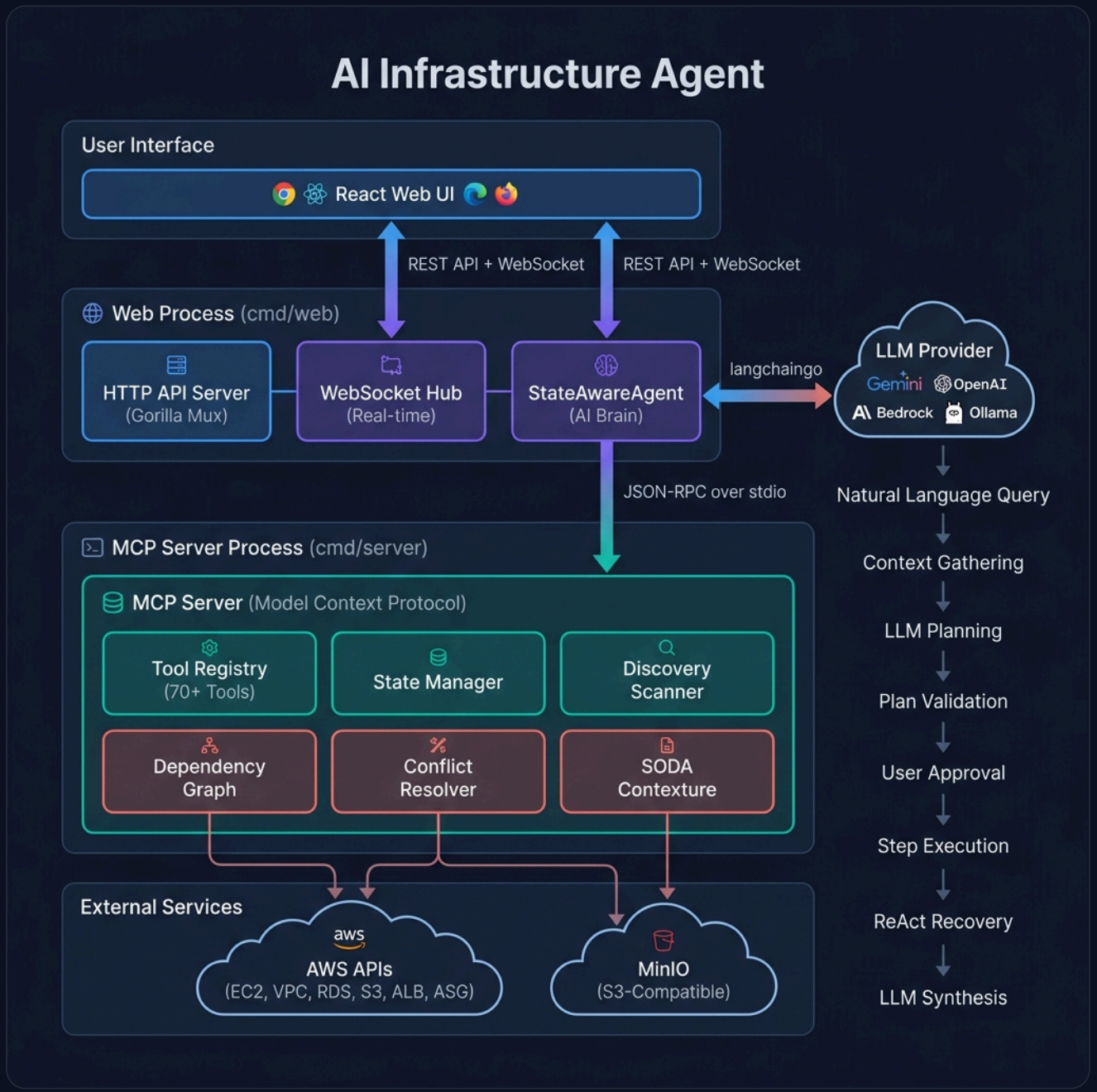
Complete System Architecture & Technical Reference

An AI-powered SRE copilot that converts natural-language requests into validated AWS execution plans via LLM + Model Context Protocol

github.com/versus-control/ai-infrastructure-agent · May 2026

1 System Overview

The AI Infrastructure Agent is a **Go-based dual-process system** that accepts natural-language infrastructure requests, converts them into validated AWS execution plans via an LLM, and executes them through a Model Context Protocol (MCP) tool server — all observable through a React Web UI with real-time WebSocket updates.



Web Process (cmd/web)

HTTP/WebSocket gateway + AI brain. Hosts the StateAwareAgent that

MCP Process (cmd/server)

Tool execution engine. Spawned as a child process, communicates over stdin/stdout JSON-RPC. Hosts 70+

orchestrates LLM calls, plan generation, and execution coordination.

AWS tools, state management, and discovery.

2 Dual-Process Architecture

Aspect	Web Process	MCP Process
Role	HTTP/WS gateway + AI brain	Tool execution engine
Entry point	`cmd/web/main.go`	`cmd/server/main.go`
Communication	Spawns MCP as child process	Reads stdin, writes stdout
Protocol	HTTP REST + WebSocket	JSON-RPC 2.0 over stdio
Lifecycle	Long-lived	Child of Web Process

The Web Process spawns the MCP Process via `go run cmd/server/main.go` (dev) or `/app/server` (Docker). They communicate over stdin/stdout pipes using JSON-RPC 2.0.

3 Package Map (14 packages)

```
pkg/
├── adapters/       # AWS resource adapters (EC2, VPC, RDS, ASG, ALB)
├── agent/          # AI agent core — LLM, plan generation, execution, recovery
├── api/            # HTTP server, REST handlers, WebSocket hub
├── aws/            # AWS SDK v2 client wrapper
├── conflict/       # Infrastructure conflict detection & resolution
├── contexture/     # SODA Contexture — S3/MinIO data landscape analysis
├── discovery/      # Live AWS resource scanning
├── graph/          # Dependency graph — topological sort, deployment levels
├── interfaces/     # Shared Go interfaces (ToolFactory, StateManager)
├── mcp/            # MCP server, resource registry, tool registration
├── state/          # Infrastructure state persistence (JSON file)
├── tools/          # 70+ MCP tool implementations + factory
├── types/          # Shared types (AWSResource, AgentDecision, etc.)
└── utilities/      # Helper functions
internal/
├── config/         # Viper-based configuration (config.yaml)
└── logging/        # Logrus-based structured logging
```

4 Request Lifecycle

Every user request passes through **8 phases**:

① NL Query → ② Context Gathering → ③ LLM Planning → ④ Validation →
⑤ User Approval → ⑥ Execution → ⑦ ReAct Recovery → ⑧ Synthesis

Phase 1 — Context Gathering

Agent calls MCP tools: `analyze-infrastructure-state`, `detect-infrastructure-conflicts`, `plan-infrastructure-deployment` to build a DecisionContext with current state, discovered resources, conflicts, and deployment order.

Phase 2 — LLM Planning

A detailed prompt is built with available MCP tools, infrastructure state, and the user request. The LLM returns a JSON decision: `{action, reasoning, confidence, executionPlan}`.

Phase 3 — Validation & Approval

The plan is validated (action types, MCP tool existence, conflict checks). A virtual "synthesis" step is appended. The plan is sent to the UI for user confirmation.

Phase 4 — Execution & Recovery

Steps execute sequentially via MCP tool calls. On failure, the ReAct loop consults the LLM for a recovery plan (up to 3 attempts). State is persisted after each successful step. A final LLM synthesis generates the SRE report.

5 Core Components

5.1 StateAwareAgent (pkg/agent/)

The central orchestrator containing the LLM client, MCP process handle, resource mappings, and configuration-driven components.

File	Responsibility
`types.go`	All agent structs: StateAwareAgent, MCPProcess, recovery types
`agent_factory.go`	LLM initialization (5 providers), agent construction
`agent_request_processor.go`	ProcessRequest() → context → LLM → parse → validate
`agent_plan_executor.go`	ExecutePlanWithReActRecovery() — step execution loop
`mcp_communication.go`	MCP subprocess management, JSON-RPC, tool calls
`react_plan_recovery_engine.go`	AI-powered failure analysis and recovery plan generation

5.2 Multi-LLM Support

Provider	Config Key	Library
Google Gemini	`GEMINI_API_KEY`	langchaingo/llms/googleai
OpenAI	`OPENAI_API_KEY`	langchaingo/llms/openai
Anthropic	`ANTHROPIC_API_KEY`	langchaingo/llms/anthropic
AWS Bedrock/Nova	AWS default creds	langchaingo/llms/bedrock
Ollama (local)	`OLLAMA_SERVER_URL`	langchaingo/llms/ollama

5.3 MCP Server (pkg/mcp/)

Implements the Model Context Protocol using `mark3labs/mcp-go`. Provides **Resources** (read-only AWS views: EC2, VPC, RDS, ASG, ALB, Subnets, Launch Templates, AMIs) and **Tools** (70+ executable actions).

5.4 Tool System (pkg/tools/)

Category	Count	Examples
EC2	9	create/list/start/stop/terminate instance, create AMI
VPC & Networking	14	create VPC/subnet/IGW/NAT/route-table, add routes
Security Groups	5	create/list/delete SG, add ingress/egress rules
ALB	7	create LB/target-group/listener, register targets
ASG	4	create/update/list ASG, launch templates
RDS	9	create/start/stop/delete DB, snapshots, subnet groups
S3	4	create/list/delete buckets, list objects
Contexture	3	analyze-data-landscape, describe-bucket, get-object-metadata
State	8	analyze/export state, save, add/update resource, graph, conflicts
Helpers	4	get AZ, get latest AMIs

5.5 State Management (pkg/state/)

- Persists infrastructure state to `./states/infrastructure-state.json`
- Tracks resources, dependencies, SHA-256 checksums, and metadata
- Supports drift detection by comparing checksums
- Atomic file writes via temp-file + rename pattern

5.6 Dependency Graph (pkg/graph/)

- Builds a DAG from resource dependencies
- **Topological sort** for deployment ordering
- **Reverse sort** for safe deletion ordering
- **Level calculation** for parallel deployment groups
- Circular dependency detection

5.7 SODA Contexture Engine (pkg/contexture/)

Extends awareness to S3/MinIO data following the **Open Context Specification (OCS)**:

- `BuildDataLandscape()` — scans all buckets, infers schemas from CSV/JSON
- `BuildBucketContext()` — per-bucket metadata: object count, sizes, formats, samples
- `GetObjectMetadata()` — per-object HEAD with content-type, etag, last-modified

6 ReAct Recovery Engine

When a plan step fails, the agent enters a **Reason + Act** recovery loop:

1. **Detect Failure** — step execution returns error
2. **Build Context** — completed steps, failed step, remaining steps, current state
3. **Consult LLM** — AI generates root cause analysis + new recovery plan
4. **Execute Recovery** — preserves completed steps, executes new/modified steps
5. **Retry or Fail** — up to 3 attempts, 30min timeout, 0.5 min confidence

7 API Surface

Method	Path	Purpose
GET	`/health`	Health check
GET	`/api/state`	Current infrastructure state
POST	`/api/agent/process`	NL request → AI decision + plan
POST	`/api/agent/execute-with-plan-recovery`	Execute approved plan with recovery
POST	`/api/discover`	Scan live AWS resources
GET	`/api/graph`	Dependency graph
GET	`/api/conflicts`	Conflict detection
POST	`/api/plan`	Deployment ordering
GET	`/api/export`	Export state JSON
GET/POST	`/api/deletion-plan`	Safe deletion planning

WebSocket Messages (`/ws`)

Type	Direction	Purpose
`state_update`	Server → Client	Periodic state refresh
`processing_started/completed`	Server → Client	AI analyzing request
`step_started/completed/failed`	Server → Client	Per-step progress
`plan_recovery_analyzing`	Server → Client	AI analyzing failure
`execution_completed`	Server → Client	Final summary + SRE report

8 Configuration

```yaml
# config.yaml
server:
  port: 3000
agent:
  provider: "gemini"
  model: "gemini-flash-latest"
  max_tokens: 8192
  temperature: 0.0
  dry_run: false
  enable_debug: true
aws:
  region: "us-west-2"
state:
  file_path: "./states/infrastructure-state.json"
web:
  port: 8080
  enable_websockets: true
```

Settings YAML files in `./settings/` drive configuration-based resource extraction:

- `field-mappings-enhanced.yaml` — maps tool responses to unified resource fields
- `resource-extraction-enhanced.yaml` — regex patterns for extracting resource IDs
- `resource-patterns-enhanced.yaml` — resource type recognition patterns

9 Data Flow Summary

```
User Query
    │
    ▼
┌──────────────────────┐
│   Context Gathering  │ ← MCP: analyze-state, detect-conflicts
└──────────┬───────────┘
           │
           ▼
┌──────────────────────┐
│   LLM Planning       │ ← Prompt with tools + state → JSON decision
└──────────┬───────────┘
           │
           ▼
┌──────────────────────┐
│   Plan Validation    │ ← Verify actions, check MCP tool existence
└──────────┬───────────┘
           │
           ▼
┌──────────────────────┐
```


User Approval
← Frontend shows plan, user confirms
Step Execution
(ReAct Recovery)
← MCP tool calls → AWS → state persistence
On failure: AI recovery → new plan → retry
LLM Synthesis
← SRE-friendly summary of execution